

Módulo 4 · Procesamiento de Lenguaje Natural aplicado a Software

NLP Aplicado a Software — Fundamentos e Investigación

Ejercicio de Clase · Ejercicio 1 de 3 · Bloques 2.1 — 2.2 — 2.3 — 2.6

Nombre del alumno/a	Angel Echenique
Fecha	26/04/2026
☐ Enlace GitHub del repositorio	

☐ **Instrucciones:** Responde con lo que hemos visto en clase. Puedes consultar tus apuntes. No hay que escribir código, ¡solo demostrar que entendiste! Algunas preguntas requieren investigación adicional.

BLOQUE 1 — ¿Qué permite el NLP en una aplicación? (2.1)

En clase vimos las capacidades principales que el NLP aporta a las aplicaciones. Completa la tabla:

Nº	Capacidad NLP	¿Qué hace exactamente?	Ejemplo concreto de uso
1	Entender texto	Interpreta el significado de un mensaje, su contexto y lo que el usuario quiere expresar.	Detectar si un usuario está haciendo una pregunta, reportando un error o pidiendo ayuda.
2	Generar respuestas coherentes	Produce texto natural, claro y adecuado al contexto de la conversación o tarea.	Un chatbot que responde dudas técnicas con tono profesional.
3	Resumir documentos	Reduce textos largos a ideas clave sin perder la información más importante.	Convertir un informe de 15 páginas en 5 puntos esenciales.
4	Detectar intención del usuario	Identifica qué quiere hacer el usuario, aunque no lo diga de forma literal.	Clasificar un mensaje como incidencia, consulta general o feedback.
5	Extraer entidades (NER)	Detecta datos concretos dentro de un texto, como nombres, fechas, lugares o cantidades.	Extraer nombre del cliente, fecha de entrega y número de pedido de un correo.

6	Buscar por significado (semántica)	Encuentra información relacionada por sentido, aunque no use las mismas palabras exactas.	Buscar "problemas de acceso" y encontrar documentos sobre "fallos de autenticación".
---	------------------------------------	---	--

1.2 — Las capas de análisis del NLP

El NLP avanzado no solo 'lee palabras'. Explica con tus propias palabras cada capa de análisis:

Tipo de análisis	¿Qué analiza exactamente?	Ejemplo práctico
Análisis sintáctico	Analiza la estructura gramatical de la oración: cómo se organizan las palabras y qué función cumplen.	Detectar sujeto, verbo y complemento en "Juan cerró la sesión".
Análisis semántico	Analiza el significado del texto y la relación entre sus palabras e ideas.	Entender que "iniciar sesión" y "hacer login" significan casi lo mismo.
Análisis pragmático	Analiza la intención real del usuario según el contexto.	Interpretar "No puedo entrar" como una solicitud de ayuda técnica.
Análisis de sentimiento	Detecta la emoción o polaridad del mensaje: positiva, negativa o neutral.	Identificar que "La app falla siempre, qué desastre" expresa sentimiento negativo.

1.3 — Completa la frase clave

"El NLP mediante APIs convierte tareas complejas de IA en simples llamadas a funciones. No necesitas ser experto en machine learning para construir aplicaciones inteligentes."

BLOQUE 2 — APIs y Técnicas NLP principales (2.2 + 2.7)

Une cada técnica NLP con su descripción, el proveedor/API recomendada y cuándo usarla:

Técnica NLP	¿Qué hace?	API recomendada	¿Cuándo usarla?
Summarization	Resume textos largos y los reduce a ideas clave sin perder el contexto importante.	GPT-4o o Claude	Cuando necesitas resumir informes, artículos, PDFs o documentación extensa.
Intent classification	Detecta qué quiere hacer el usuario, como preguntar, reclamar, pedir soporte o solicitar una acción.	Hugging Face BERT u OpenAI	Cuando necesitas clasificar mensajes, tickets o consultas por intención.
Embeddings	Convierte el texto en vectores numéricos para comparar significado en vez de palabras exactas.	OpenAI Embeddings o Cohere	Cuando quieres búsqueda semántica, RAG, FAQs inteligentes o similitud entre textos.
NER (Named Entity Recognition)	Extrae entidades concretas como nombres, fechas, lugares, organizaciones o cantidades.	spaCy, Hugging Face NER u OpenAI	Cuando necesitas sacar información específica de contratos, correos, formularios o documentos.
Chat completions	Genera respuestas naturales, coherentes y adaptadas al contexto de la conversación.	GPT-4o o Claude	Cuando construyes chatbots, asistentes, tutores o soporte conversacional.
Translation	Traduce texto manteniendo significado, tono y precisión técnica.	Gemini o GPT-4o	Cuando necesitas traducir documentación, mensajes o contenido técnico sin perder matices.

2.2 — Verdadero o Falso: APIs de NLP

Marca V (Verdadero) o F (Falso) y justifica brevemente las que sean Falsas:

V / F	Afirmación	Justificación si es Falsa
F	Los embeddings buscan coincidencias exactas de palabras en el texto.	Es falsa porque los embeddings buscan similitud de significado, no coincidencias literales.
V	Claude es recomendado para tareas que requieren contexto largo como análisis de documentos extensos.	
F	La temperature debe ser alta (0.8-1.0) para tareas de extracción de entidades.	Es falsas porque en extraccion conviene una temperature baja para obtener respuestas mas estables y p`recisas.
V	La búsqueda semántica puede encontrar 'fallos de autenticación' al buscar 'problemas de login'.	
F	Hugging Face solo permite usar modelos en la nube, no localmente.	Es falsas porque mucho modelos de hugging Face puede descargarse y ejecutarse localmente.
V	GPT-4o-mini es más adecuado que GPT-4o para clasificación simple de tickets.	

2.3 — ¿Qué proveedor/técnica elegirías?

Para cada caso de uso, indica qué técnica NLP y qué proveedor/API usarías, justificando brevemente:

Caso de uso	Técnica NLP	Proveedor / API	¿Por qué?
Analizar 1000 reseñas de clientes y detectar quejas recurrentes	Análisis de sentimiento + clasificación	Hugging Face o OpenAI	Permite detectar polaridad, temas repetidos y clasificar opiniones de forma rápida y escalable.

Buscar documentación técnica usando lenguaje natural	Embeddings	OpenAI Embeddings	Porque la búsqueda semántica encuentra contenido relacionado aunque no coincidan las palabras exactas.
Chatbot de soporte que recuerde el contexto de la conversación	Chat completions	GPT-4o o Claude	Porque puede mantener contexto, generar respuestas coherentes y adaptarse al historial del chat.
Traducir documentación de API manteniendo términos técnicos	Translation	GPT-4o o Gemini	Porque ofrecen traducción contextual y respetan mejor la terminología técnica.
Extraer nombres, fechas y montos de contratos en PDF	NER + análisis de documentos	spaCy, Hugging Face NER o OpenAI	Porque permiten localizar entidades concretas dentro de textos largos y estructurar la información extraída.

BLOQUE 3 — El Flujo NLP en una App (2.3)

Vimos que el flujo de NLP tiene 7 fases. Define cada fase con tus palabras e indica su importancia:

Fase	Nombre	¿Qué se hace?	¿Por qué es necesaria?
1	Recepción del texto	La aplicación recibe el mensaje, documento, reseña o consulta del usuario.	Porque todo el proceso empieza con la entrada de información que se va a analizar.
2	Preprocesamiento	Se limpia y normaliza el texto: quitar HTML, espacios extra, caracteres raros o recortar si es demasiado largo.	Porque mejora la calidad del input y evita errores o costes innecesarios.

3	Construcción del prompt	Se define el system prompt, el contexto, las instrucciones y el texto del usuario.	Porque un prompt claro hace que la respuesta sea más precisa, útil y consistente.
4	Llamada a la API	La app envía la petición al modelo con parámetros como <code>model</code> , <code>temperature</code> o <code>max_tokens</code> .	Porque es el momento en que el modelo procesa realmente el texto.
5	Validación de la respuesta	Se comprueba si la API respondió bien y si el formato devuelto es correcto, por ejemplo JSON válido.	Porque no se debe confiar ciegamente en la salida de la IA antes de usarla en la app.
6	Transformación de la respuesta	La salida se adapta al formato interno de la aplicación, por ejemplo objetos, enums o campos de base de datos.	Porque la respuesta del modelo debe integrarse de forma útil en el sistema.
7	Acción basada en el resultado	La aplicación decide qué hacer con el análisis: mostrar algo, guardar datos, redirigir o escalar un caso.	Porque el valor real del NLP está en activar una acción útil dentro de la app.

3.2 — Investiga: Parámetros clave de la API

Investiga y define con tus propias palabras cada parámetro. Incluye un valor de ejemplo y cuándo usarlo:

Parámetro	¿Qué controla?	Valor de ejemplo	¿Cuándo usar ese valor?
<code>temperature</code>	Controla cuánta variación o creatividad tendrá la respuesta del modelo.	<code>0.1</code>	Cuando quieres respuestas estables y precisas, por ejemplo en clasificación, extracción o análisis.
<code>max_tokens</code>	Limita la longitud máxima de la respuesta generada.	<code>300</code>	Cuando quieres controlar costes y evitar respuestas demasiado largas.
<code>model</code>	Define qué modelo se va a utilizar según	<code>gpt-4o-mini</code>	Cuando la tarea es simple, como clasificar tickets o extraer datos

	complejidad, coste y velocidad.		básicos, y quieres rapidez y menor coste.
<code>response_format</code>	Indica el formato esperado de salida, como JSON estructurado.	<code>json</code>	Cuando la aplicación necesita procesar la respuesta automáticamente sin texto libre.
<code>system prompt</code>	Marca el rol, tono, reglas y formato que debe seguir el modelo.	"Eres un analista técnico. Responde solo en JSON."	Cuando necesitas controlar con claridad el comportamiento del modelo y evitar respuestas ambiguas.

BLOQUE 4 — Búsqueda Semántica y Embeddings (2.6)

Los embeddings son un concepto clave para búsqueda semántica. Responde:

Pregunta	Tu respuesta
¿Qué es un embedding en términos simples?	Un embedding es una representación numérica de un texto en forma de vector. Sirve para que una máquina compare significados entre textos, no solo palabras exactas.
¿En qué se diferencia buscar por palabras de buscar por embeddings?	La búsqueda por palabras busca coincidencias literales o muy parecidas. La búsqueda por embeddings compara significado, así que puede encontrar textos relacionados aunque usen palabras distintas.
¿Qué modelo se recomienda para embeddings simples y por qué?	<code>text-embedding-3-small</code> , porque es rápido, más económico y suficiente para tareas como búsqueda semántica, similitud y clasificación básica.
¿Qué es RAG (Retrieval Augmented Generation)?	Es una técnica que primero recupera información relevante desde una base de conocimiento, normalmente con embeddings, y luego usa esa información como contexto para generar una respuesta más precisa.

Pon un ejemplo de dos frases con palabras distintas pero significado similar	"No puedo iniciar sesión en mi cuenta" y "Tengo problemas para acceder a mi usuario".
--	---

4.2 — Investiga: Casos de uso reales de embeddings

Busca información y describe un caso de uso real de embeddings en producción para cada categoría:

Categoría	Empresa / Producto real	¿Cómo usa los embeddings?
E-commerce	Wayfair	Wayfair usa modelos de OpenAI en producción para mejorar la calidad de su catálogo y enriquecer atributos de productos a gran escala. Eso ayuda a que los productos se descubran mejor en búsqueda, recomendaciones y merchandising.
Soporte técnico	Zendesk Semantic Search	Zendesk usa búsqueda semántica en su help center para que usuarios y agentes encuentren artículos por intención y contexto, no solo por palabras exactas.
Plataforma legal	Harvey Vault	Harvey permite buscar y consultar acuerdos, escritos y documentos legales dentro de su "Vault". Por la forma en que describe esa búsqueda sobre documentos y consultas en lenguaje natural, se infiere que usa recuperación semántica basada en embeddings o tecnología vectorial similar.
Motor de búsqueda	Elastic Vector Search	Elastic ofrece búsqueda vectorial para experiencias de búsqueda semántica en producción. Usa embeddings para recuperar resultados por significado y similitud, no solo por coincidencia textual.

Los embeddings son clave porque convierten texto en vectores numéricos que conservan significado. Gracias a eso, una aplicación puede comparar consultas y documentos por cercanía semántica, algo que la búsqueda tradicional por palabras no logra tan bien.

Fuentes

- OpenAI Embeddings Docs
- Wayfair boosts catalog accuracy and support speed with OpenAI
- Zendesk: About semantic search and how it works
- Harvey Mobile / Vault
- Elastic Vector Search

BLOQUE 5 — Pregunta de Síntesis e Investigación

Un compañero/a dice:

"Prefiero usar búsqueda por palabras clave (LIKE en SQL) antes que embeddings. Es más rápido y no necesito API."

¿Estás de acuerdo? Argumenta tu respuesta con al menos 3 razones concretas. Menciona al menos un caso donde la búsqueda por palabras SÍ sería suficiente:

No estoy completamente de acuerdo con esa afirmación. La búsqueda por palabras clave con LIKE en SQL puede ser útil en casos simples, pero en muchos escenarios reales se queda corta frente a los embeddings. Los embeddings no reemplazan siempre a la búsqueda tradicional, pero sí resuelven problemas que las palabras clave no pueden manejar bien.

La primera razón es que la búsqueda por palabras depende demasiado de la coincidencia literal. Si el usuario escribe "problemas de acceso" y en la documentación aparece "fallos de autenticación", una búsqueda con palabras clave puede no encontrar nada relevante o devolver resultados pobres. En cambio, los embeddings comparan el significado de los textos, por lo que pueden relacionar expresiones distintas que hablan del mismo concepto.

La segunda razón es que los embeddings manejan mejor el lenguaje natural real de los usuarios. Las personas no siempre usan los términos exactos que aparecen en una base de datos o en una documentación técnica. Pueden usar sinónimos, frases incompletas,

reformulaciones o lenguaje poco preciso. La búsqueda semántica entiende mejor esa intención, mientras que una búsqueda literal obliga al usuario a "adivinar" las palabras correctas.

La tercera razón es que los embeddings son mucho más potentes cuando trabajamos con grandes volúmenes de información no estructurada, como contratos, manuales, tickets, reseñas o artículos de ayuda. En esos casos no solo importa encontrar una palabra, sino recuperar el fragmento más relevante por contexto. Esto es clave en sistemas modernos como buscadores inteligentes, soporte técnico, RAG o análisis documental.

Además, aunque un compañero diga que "no necesita API", esa ventaja no siempre compensa la pérdida de calidad. Una búsqueda tradicional puede ser más simple y barata al principio, sí, pero si devuelve resultados poco útiles, el coste real aparece después en forma de mala experiencia de usuario, más tiempo perdido y menor precisión.

Dicho esto, sí hay casos donde la búsqueda por palabras clave es suficiente. Por ejemplo, cuando queremos encontrar un dato exacto y estable, como un número de factura, un DNI, un código postal, un ID de pedido o un nombre de endpoint concreto en documentación técnica. En esos casos, la coincidencia literal puede ser incluso más adecuada que una búsqueda semántica.

En conclusión, no elegiría solo palabras clave "porque es más rápido". Elegiría la técnica según la necesidad. Para datos exactos, filtros simples o búsquedas estructuradas, LIKE o búsqueda textual clásica puede bastar. Pero para entender intención, encontrar información relacionada y buscar por significado, los embeddings ofrecen una solución mucho más potente, flexible y útil en aplicaciones modernas.